

PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility

For this task, I created the uptime command to show how long xv6 has been running. I used the existing uptime() system call to get the number of clock ticks since the system started. I also added the program to the Makefile so it could be executed from the xv6 shell. The screenshot below shows that the command is working correctly.

```
init: starting sh
$ uptime
up 46 clock ticks
$
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

For this task, I created the time1 command to measure the elapsed time of another program. The program records the uptime before creating the child process, then uses fork() and exec() to run the command. After the child finishes, it records the uptime again and calculates the elapsed time. I tested it with the sleep command and the results showed that the elapsed time was measured correctly.

```
$ time1 sleep 10
elapsed time: 10 ticks
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

For this task, I added a cputime field to the process structure so xv6 can keep track of how much CPU time each process uses. I initialized the value to zero when a process is created and updated it whenever the process uses a timer slice. I made the changes in proc.h, proc.c, and trap.c.

```
kernel/proc.h kernel/proc.c kernel/trap.c
kernel/proc.h:91:  uint cputime;          // CPU time used by process
kernel/proc.c:150:  p->cputime = 0;
kernel/proc.c:453:  r.cputime = pp->cputime;
kernel/trap.c:86:   p->cputime++;
kernel/trap.c:161:  myproc()->cputime++;
daniels12398@daniels12398-VMware-Virtual-Platform:~/xv6-riscv$
```

```

34 // give up the CPU if this is a timer interrupt.
35 if (which_dev == 2){
36     p->cputime++;
37
38     yield();
39 }
40
41 prepare_return();
42 panic("kerneltrap");
43 }
44
45 // give up the CPU if this is a timer interrupt.
46 if (which_dev == 2 && myproc() != 0){
47     myproc()->cputime++;
48 }

```

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

For this task, I implemented a new wait2() system call based on the existing wait() system call. The main difference is that wait2() also returns resource usage information from the child process. I created the rusage structure with a cputime field and connected the new system call between user space and the kernel. This required changes to the syscall definitions, user declarations, process code, and syscall handling code.

```

daniels12398@daniels12398-VMware-Virtual-Platform:~/xv6-riscv$ cat kernel/pstat.h
#include "types.h"

struct rusage {
    uint cputime;
};

daniels12398@daniels12398-VMware-Virtual-Platform:~/xv6-riscv$ grep -n "wait2"
user/user.h:9: int wait2(int*, struct rusage*);
user/usys.pl:27: entry("wait2");
kernel/syscall.h:24: #define SYS_wait2 23
kernel/syscall.c:87: extern uint64 sys_wait2(void);
kernel/syscall.c:115: [SYS_wait2] = sys_wait2,
kernel/sysproc.c:39: sys_wait2(void)
kernel/sysproc.c:47: return kwait2(p, r);
kernel/defs.h:101: int kwait2(uint64, uint64);
kernel/proc.c:423: kwait2(uint64 addr, uint64 raddr)

```

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

For this task, I created the time command using the new wait2() system call. The command measures both the elapsed time and the CPU time used by the child process. It then calculates the CPU percentage by comparing the CPU time with the elapsed time. I tested the command with sleep, matmul, and two concurrent matmul processes. The results showed the expected difference between a sleeping process and a CPU-intensive process.

```

$ time matmul
Time: 9 ticks
elapsed time: 9 ticks, cpu time: 9 ticks, 100% CPU
$

```

```
$ time matmul & time matmul
Time: 14 ticks
elapsed time: 15 ticks, cpu time: 7 ticks, 46% CPU
Time: 14 ticks
elapsed time: 14 ticks, cpu time: 8 ticks, 57% CPU
$ 
init: starting sh
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ 
```

When I ran sleep 10, the CPU usage was 0% because the process spent most of its time sleeping. When I ran matmul by itself, it used 100% CPU because it was continuously performing calculations. When I ran two matmul processes at the same time, each process received approximately half of the CPU time. In my test, the results were 50% and 53%.

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment helped me understand how user programs and system calls work together in xv6. Before this assignment, I did not fully understand all the steps needed to add a new system call. Implementing wait2() helped me see how information can be passed from the kernel back to a user program. I also learned more about process scheduling and how timer interrupts can be used to keep track of CPU time.

One of the most difficult parts was debugging the changes in trap.c. A problem with the placement of yield() caused a kernel trap during boot. Fixing that problem helped me understand why small changes in kernel code can affect the entire operating system. Testing time with sleep and matmul also helped me understand the difference between elapsed time and actual CPU time.